

L1 Technical Questions:

1. Core Java — Must Prepare

1. Explain the major features introduced from **Java 8 to Java 21**.
2. What is the difference between **HashMap**, **ConcurrentHashMap** and **Hashtable**?
3. How does HashMap internally work?
4. What happens when two keys have the same hash code?
5. Explain `equals()` and `hashCode()` contract.
6. Difference between **ArrayList** and **LinkedList**.
7. Difference between **Comparable** and **Comparator**.
8. Explain Java Streams. When would you avoid using streams?
9. Difference between `map()`, `flatMap()` and `filter()`.
10. What is Optional? What problems does it solve?
11. Explain functional interfaces and lambda expressions.
12. Difference between **checked** and **unchecked exceptions**.
13. How do you create a custom exception?
14. Explain **final**, **finally** and **finalize**.
15. What is immutability? How would you create an immutable class?
16. Explain multithreading in Java.
17. Difference between `synchronized`, `volatile`, `Lock` and Atomic classes.
18. What is deadlock? How do you identify and prevent it?
19. What is thread pool? Why use `ExecutorService`?
20. Explain JVM memory — **Heap**, **Stack**, **Metaspace**.
21. How does Garbage Collection work?
22. What causes `OutOfMemoryError`?
23. How would you troubleshoot high CPU usage in a Java application?
24. What are Java records and sealed classes?
25. What Java 17/21 features have you actually used in production?

Scenario

Your API suddenly starts consuming 90% CPU in production. What will you do?

Expected discussion:

- Check monitoring/logs
 - Identify affected instances
 - Thread dump
 - CPU-intensive thread identification
 - Check recent deployment
 - DB/API dependency
 - Rollback if required
 - Root-cause analysis
 - Permanent fix
-

2. Spring Boot — Very High Priority

1. Explain Spring Boot architecture.
 2. Difference between **Spring** and **Spring Boot**.
 3. What is Dependency Injection?
 4. Explain IoC container.
 5. Difference between `@Component`, `@Service`, `@Repository`, `@Controller` and `@RestController`.
 6. Explain Spring Bean lifecycle.
 7. What are different bean scopes?
 8. Constructor injection vs field injection.
 9. Explain `@Autowired`.
 10. What is Spring Boot auto-configuration?
 11. What is `application.yml/application.properties` used for?
 12. How do you manage environment-specific configurations?
 13. Explain Spring Profiles.
 14. How do you secure sensitive properties such as DB passwords?
 15. Explain Spring Boot Actuator.
 16. How do you implement global exception handling?
 17. Explain `@ControllerAdvice` and `@ExceptionHandler`.
 18. Difference between `@RequestParam`, `@PathVariable` and `@RequestBody`.
 19. How do you implement validation in Spring Boot?
 20. Explain `@Valid` and `@Validated`.
 21. How do you implement logging in a Spring Boot application?
 22. How do you handle correlation IDs?
 23. How do you implement API versioning?
 24. How do you handle timeout and retry?
 25. How do you improve Spring Boot application performance?
-

3. Spring MVC / REST API

1. Explain the request flow in Spring MVC.
2. What is REST?
3. Difference between **GET**, **POST**, **PUT**, **PATCH** and **DELETE**.
4. What HTTP status codes would you use for:
 - Successful creation?
 - Bad request?
 - Unauthorized?
 - Forbidden?
 - Resource not found?
 - Internal server error?
5. What is idempotency?
6. Why is idempotency important in banking applications?
7. How would you design a REST API for order management?

8. How would you handle duplicate requests?
9. How do you implement pagination?
10. How do you handle API backward compatibility?
11. How do you secure REST APIs?
12. JWT vs OAuth2?
13. How do you prevent sensitive information from appearing in logs?

Scenario

A client sends the same order request twice because of a network timeout. How will you make sure two orders aren't created?

A strong answer should discuss **idempotency keys**, unique business/reference IDs, database constraints and transactional handling.

4. Spring Data JPA / Hibernate — Must Prepare

1. What is JPA?
2. JPA vs Hibernate.
3. Explain entity lifecycle.
4. Difference between:
 - @OneToOne
 - @OneToMany
 - @ManyToOne
 - @ManyToMany
5. Lazy vs eager loading.
6. What is the **N+1 query problem**?
7. How do you solve N+1?
8. Difference between `save()`, `saveAndFlush()`.
9. What is dirty checking?
10. Explain first-level and second-level cache.
11. What is cascading?
12. What is orphan removal?
13. Explain `@Transactional`.
14. What happens when an exception occurs inside a transaction?
15. Transaction propagation types.
16. Optimistic vs pessimistic locking.
17. JPQL vs native query.
18. How do you optimize a slow JPA query?
19. What indexes would you create?
20. How would you handle millions of records?

Scenario

A portfolio API was taking 2 seconds earlier but suddenly takes 15 seconds after the data volume increased. How would you investigate?

Discuss:

Application → Hibernate/JPA → SQL → Execution Plan → Indexes → DB locks → Network → Pagination → Query optimization

5. Spring Batch — JD Specifically Mentions It

1. What is Spring Batch?
2. Explain **Job**, **JobInstance**, **JobExecution** and **Step**.
3. Chunk processing vs tasklet.
4. What are ItemReader, ItemProcessor and ItemWriter?
5. How does chunk size affect performance?
6. How do you handle failed records?
7. What is retry and skip policy?
8. How do you restart a failed batch?
9. How do you prevent duplicate processing?
10. How do you process millions of records efficiently?
11. How would you monitor a batch job?
12. How would you schedule batch jobs?

Scenario

A batch processes 10 million portfolio transactions overnight and fails after processing 7 million. What would you do?

They'll expect concepts such as **restartability**, **checkpointing**, **chunk processing**, **skip/retry**, **idempotency** and **transaction management**.

6. AOP — Must Know

1. What is AOP?
2. What are:
 - Aspect
 - Advice
 - Pointcut
 - Join Point
3. Difference between **@Before**, **@After**, **@AfterReturning**, **@AfterThrowing** and **@Around**.
4. Real-world use cases of AOP.
5. How would you use AOP for logging?
6. How can AOP be used for auditing?

7. How does Spring AOP work internally?
8. What are limitations of Spring AOP?

Scenario

BNP wants every transaction-related service call to be audited without adding audit code into every method. How would you implement it?

7. Database — Oracle + PostgreSQL

1. Difference between Oracle and PostgreSQL.
2. Explain primary key vs unique key.
3. What is normalization?
4. What is indexing?
5. Clustered vs non-clustered indexes.
6. What makes a query slow?
7. How do you troubleshoot a slow SQL query?
8. Explain joins.
9. Inner join vs left join.
10. What is a composite index?
11. When can an index actually hurt performance?
12. Explain transactions and ACID properties.
13. What are isolation levels?
14. What is database deadlock?
15. How do you handle concurrent updates?
16. What is optimistic locking?
17. Explain stored procedures.
18. How do you migrate large database tables safely?
19. How would you handle database connection pool exhaustion?
20. How do you prevent SQL injection?

Scenario

Production DB has 500 connections, but suddenly all connections are occupied and APIs are failing. What would you check?

Talk about:

- Connection pool configuration
- Long-running queries
- Connection leaks
- Unclosed transactions
- DB locks
- Slow APIs
- Pool metrics
- DB monitoring

- Application logs
-

8. Architecture / System Design — VERY IMPORTANT for 9–12 Years

Because the JD says:

"Analyze, discuss, document solution architecture and design"

Expect senior-level questions.

Questions

1. Explain the architecture of your current application.
 2. Why did you choose microservices?
 3. Microservices vs monolith?
 4. How do services communicate?
 5. REST vs messaging.
 6. How do you handle distributed transactions?
 7. Explain API Gateway.
 8. What is service discovery?
 9. What is circuit breaker?
 10. Explain retry and timeout strategies.
 11. How do you make a service highly available?
 12. How do you scale a Java application?
 13. Horizontal vs vertical scaling.
 14. How do you design for fault tolerance?
 15. How do you handle caching?
 16. Redis use cases?
 17. How do you handle concurrent requests?
 18. How do you design secure APIs?
 19. How do you implement centralized logging?
 20. Explain observability: **logs, metrics and traces**.
-

9. Architecture Scenario Questions

Scenario 1 — Portfolio Analysis

Design a system that receives portfolio information and provides portfolio analysis to private-banking clients.

They may ask:

- APIs?
 - Database?
 - Caching?
 - Authentication?
 - Scalability?
 - Error handling?
 - Audit?
 - Performance?
 - Security?
-

Scenario 2 — Order Management

Design an Order Management API where a client submits buy/sell orders.

Be prepared to explain:

None

Client



API Gateway



Order Service



Validation



Business Rules



Database



External Trading/Wealth System



Response

Then discuss:

- Idempotency
 - Transaction management
 - Concurrency
 - Audit trail
 - Security
 - Failure handling
 - Retry
 - Monitoring
-

Scenario 3 — Production Failure

An order API is returning 500 errors for some customers but working for others. How will you investigate?

A senior answer should follow:

Identify → Reproduce → Logs → Correlation ID → DB → External dependency → Recent deployment → Metrics → Fix/Rollback → RCA

10. L3 Production Support Questions

This JD explicitly says **production L3 developer-level support**, so prepare these seriously.

1. What is your experience in L3 production support?
2. How do you investigate production issues?
3. What is an RCA?
4. How do you differentiate application vs DB vs infrastructure issues?
5. How do you handle Sev-1 incidents?
6. What information do you collect before escalating?
7. How do you analyze application logs?
8. What metrics do you monitor?
9. How do you troubleshoot memory leaks?
10. How do you troubleshoot API latency?
11. What would you do after a failed production deployment?
12. How do you perform rollback?
13. How do you ensure production fixes don't introduce regression?
14. How do you handle a recurring production issue?
15. Tell me about a critical production incident you resolved.

Very likely scenario:

Production issue occurs at 2 AM and business users report that transactions are failing. Walk me through exactly what you would do.

11. Tomcat

1. What is Apache Tomcat?
2. How does Tomcat process an HTTP request?
3. What is a thread pool in Tomcat?
4. What happens if Tomcat reaches maximum threads?
5. How do you tune Tomcat?
6. How do you deploy a Spring Boot application to Tomcat?

7. Embedded Tomcat vs standalone Tomcat.
 8. How do you troubleshoot Tomcat memory issues?
 9. What causes connection timeout?
 10. How do you troubleshoot 502/503 errors?
-

12. Angular — Good to Have

Since frontend is secondary, expect basic-to-intermediate questions.

1. What is Angular?
2. Angular component lifecycle.
3. What are services?
4. Dependency injection in Angular.
5. Observable vs Promise.
6. What is RxJS?
7. What are HTTP interceptors?
8. How do you handle API errors?
9. Reactive forms vs template-driven forms.
10. How does Angular communicate with Spring Boot?
11. How do you secure frontend API calls?
12. What is lazy loading?

They probably won't go as deep as Java/Spring.

13. Testing — Important

1. Unit testing vs integration testing.
2. JUnit 5 features.
3. Mockito.
4. Mock vs Spy.
5. `@Mock` vs `@InjectMocks`.
6. How do you test REST APIs?
7. What is Test-Driven Development?
8. What is Behaviour-Driven Development?
9. How do you test database interactions?
10. How do you test external API failures?
11. How do you achieve meaningful test coverage?
12. What test cases would you write for an order API?

Scenario

An API works correctly in development but fails in integration testing. How do you investigate?

14. Banking / Wealth Management — Very Important

Because BNP's JD specifically mentions **Private Banking / Wealth Management**, learn the terminology.

Questions they may ask:

1. What do you understand by Wealth Management?
2. What is Private Banking?
3. What is Portfolio Management?
4. What is Portfolio Analysis?
5. What is Asset Management?
6. What is Order Management?
7. What is an investment order?
8. Buy order vs sell order.
9. What is an investment portfolio?
10. What are financial instruments?
11. What are equities, bonds and mutual funds?
12. What is asset allocation?
13. What is portfolio valuation?
14. What is P&L?
15. What is a transaction lifecycle?
16. Why is auditability important in banking applications?
17. What is KYC?
18. What is AML?
19. What is regulatory compliance?
20. Why are security and data confidentiality particularly important in private banking?

15. Temenos Wealth Front Office Suite

This is a **big advantage/preferred** item, so if the candidate has experience, BNP may probe it.

Possible questions:

1. What is Temenos Wealth Front Office?
2. What functionality does it provide?
3. Have you worked with Portfolio Management?
4. Have you worked with Order Management?
5. Have you worked with Advisory?
6. How did your Java application integrate with Temenos?
7. What APIs/interfaces did you use?

8. How did you handle data synchronization?
9. What challenges did you face integrating with Temenos?
10. How did you troubleshoot integration failures?

If the candidate has never worked with Temenos, don't bluff. Explain the relevant wealth-management integration experience and say you're familiar with the functional concepts but haven't worked directly on the suite.

16. Agile / Scrum

1. Explain Scrum.
 2. What happens in Sprint Planning?
 3. What is Daily Scrum?
 4. Sprint Review vs Retrospective.
 5. What is a user story?
 6. What is Definition of Done?
 7. How do you estimate tasks?
 8. How do you handle changing requirements mid-sprint?
 9. How do developers collaborate with architects/testers/business teams?
 10. Tell me about a technical disagreement with an architect.
 11. How do you communicate technical risks?
-

17. Vendor Management / Collaboration

This JD specifically says:

"Engage with external vendor consultancy development and delivery team"

So expect:

1. Have you worked with external vendors?
 2. How do you review vendor code?
 3. How do you ensure vendor deliverables meet standards?
 4. How do you handle a vendor missing a deadline?
 5. How do you communicate technical requirements to an external team?
 6. How do you handle disagreements with vendor developers?
 7. How do you ensure code quality across multiple teams?
-

18. POC / New Technology Questions

Because the JD mentions **POC for new frameworks**, expect:

1. Tell me about a POC you performed.
2. Why did you choose that technology?
3. What alternatives did you evaluate?
4. How did you compare pros and cons?
5. What criteria do you use before introducing a new framework?
6. How do you assess security risks?
7. How do you assess performance?
8. How do you convince architects/business teams to adopt a technology?

Scenario

Your architect asks you to evaluate a new Java framework. How would you decide whether BNP should adopt it?

Discuss:

POC → Performance → Security → Maintainability → Compatibility → Licensing → Developer skill → Operational impact → Cost → Recommendation

19. BNP-Specific Scenario Questions

These are the ones I'd especially prepare for this JD:

Scenario A

A private-banking client submits an order, but the external system times out after the request is sent. How do you determine whether the order was actually processed?

Scenario B

Two users simultaneously update the same portfolio. How do you prevent inconsistent data?

Scenario C

The portfolio calculation API is suddenly taking 10 seconds instead of 1 second. How do you troubleshoot it?

Scenario D

A batch job processes financial transactions overnight and fails halfway through. How do you restart it without duplicating transactions?

Scenario E

A production deployment causes 20% of requests to fail. What steps do you take?

Scenario F

The database query works with 1 lakh records but becomes extremely slow with 1 crore records. What changes would you consider?

Scenario G

An external vendor delivers code that doesn't meet your team's coding/security standards. What do you do?

Scenario H

You need to introduce a new framework into a highly regulated banking application. What factors do you evaluate?

20. Project-Based Questions — VERY LIKELY

For a senior candidate, the interviewer may spend **20–30 minutes** just drilling into the candidate's project.

Prepare these answers extremely well:

- Explain your current project.
- What is the business purpose?
- What is your role?
- Explain the architecture.
- Why Java/Spring Boot?
- How many microservices?
- What databases?
- How do services communicate?
- What APIs have you developed?
- Biggest technical challenge?
- Biggest production issue?
- How did you solve it?
- What performance improvement did you implement?
- How did you handle security?
- How did you handle database optimization?
- How did you perform code reviews?
- How did you work with QA?
- How did you work with architects?
- Have you mentored juniors?
- Have you handled vendors?
- What technical decision did **you personally** make?

Most important

For a **9–12 year candidate**, don't answer everything as:

"My team did..."

Instead clearly separate:

"The team implemented X, and my responsibility was Y. I personally designed/implemented Z."

That distinction is very important in a senior L1.

★ **Priority for L1 Preparation**

I'd prepare in this order:

Priority	Area
● Very High	Core Java 11–21
● Very High	Spring Boot 3.x
● Very High	REST/Microservices
● Very High	JPA/Hibernate
● Very High	Oracle/PostgreSQL
● Very High	Architecture/System Design
● Very High	Production/L3 scenarios
● High	Spring Batch
● High	AOP
● High	Testing/JUnit/Mockito
● High	Banking/Wealth Management
● Medium	Tomcat
● Medium	Angular
● Medium	Agile/Scrum
● Medium	Vendor collaboration
● Medium	Temenos